

**Embedded
Online
Conference**

www.embeddedonlineconference.com

Object-Oriented Programming (OOP) in C

Nathan Jones

AGENDA

1

Why and what is it?

2

Abstraction/Encapsulation

- Singletons
- Opaque structs

3

Build-time Polymorphism

- Using the linker
- Using the preprocessor

4

Run-time Polymorphism

- Function pointers

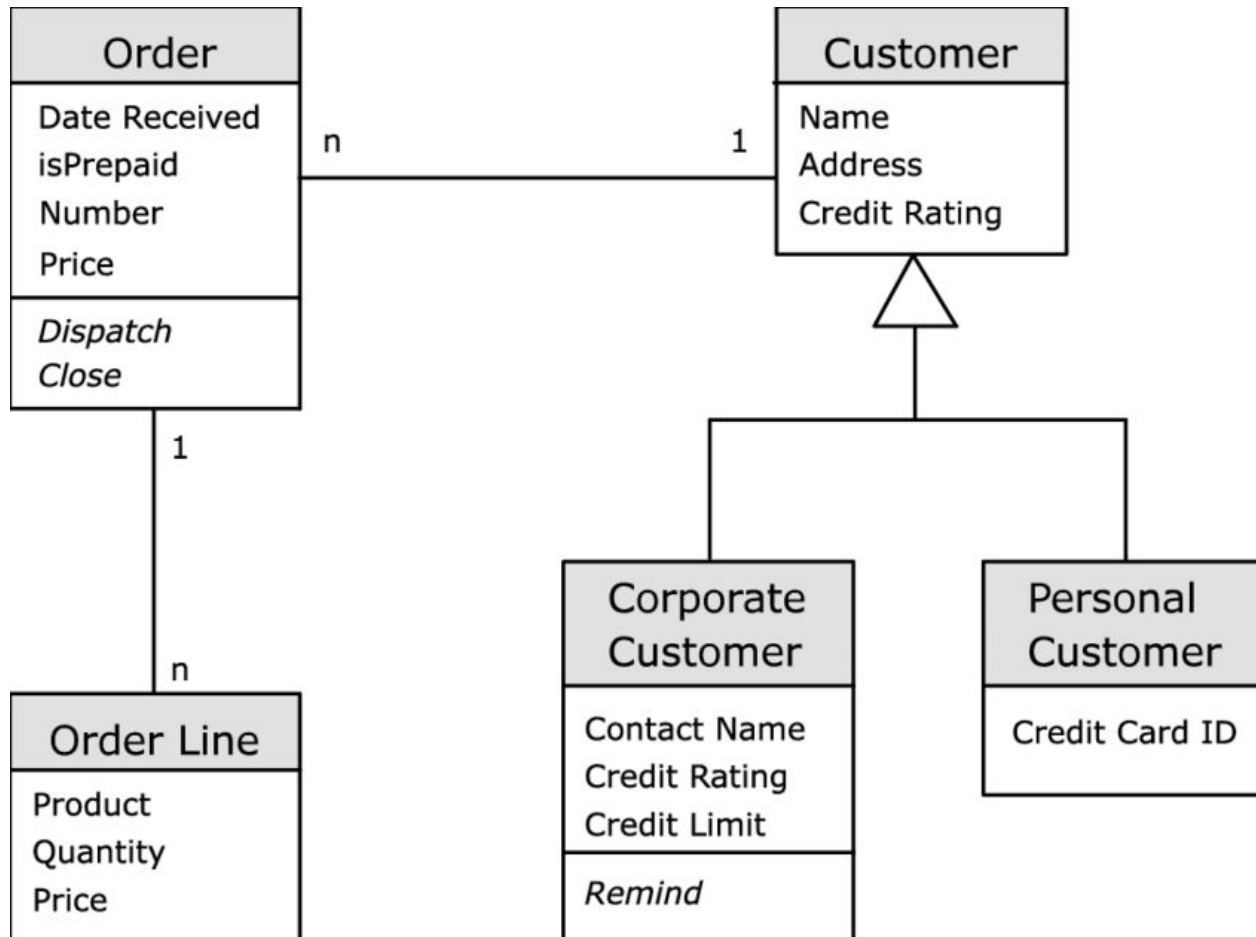
5

Inheritance

- Matroshka structs
- Manual type system

[https://github.com/nathancharlesjones/
Comparison-of-OOP-techniques-in-C](https://github.com/nathancharlesjones/Comparison-of-OOP-techniques-in-C)

Why and What is it?



Why and What is it?

Abstraction/
Encapsulation

(Build-time)
Polymorphism

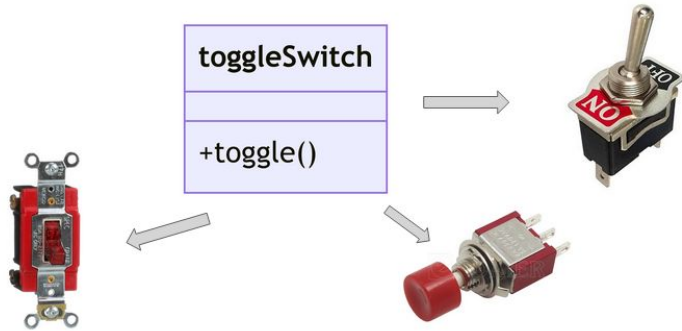
(Run-time)
Polymorphism

Inheritance

Why and What is it?

Abstraction/
Encapsulation

(Build-time)
Polymorphism

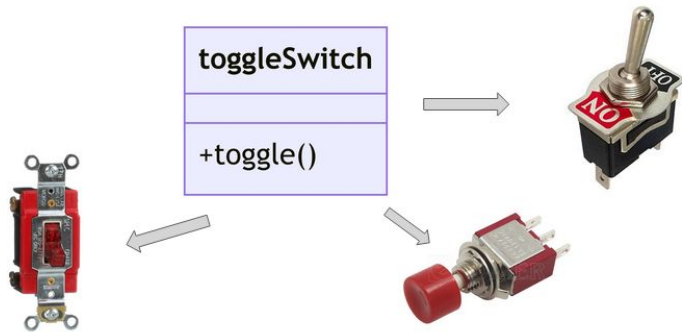


(Run-time)
Polymorphism

Inheritance

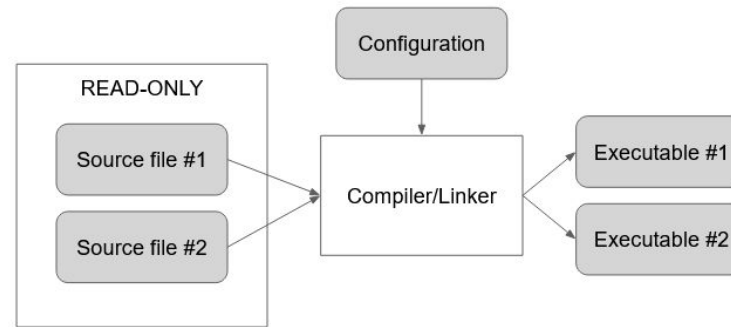
Why and What is it?

Abstraction/
Encapsulation



(Run-time)
Polymorphism

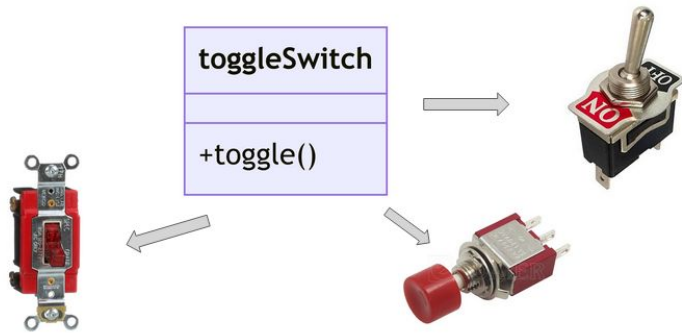
(Build-time)
Polymorphism



Inheritance

Why and What is it?

Abstraction/
Encapsulation

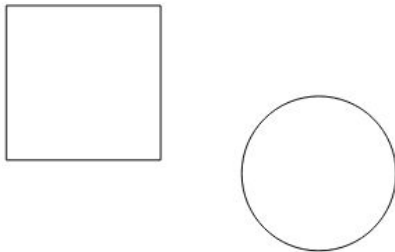


(Run-time)
Polymorphism

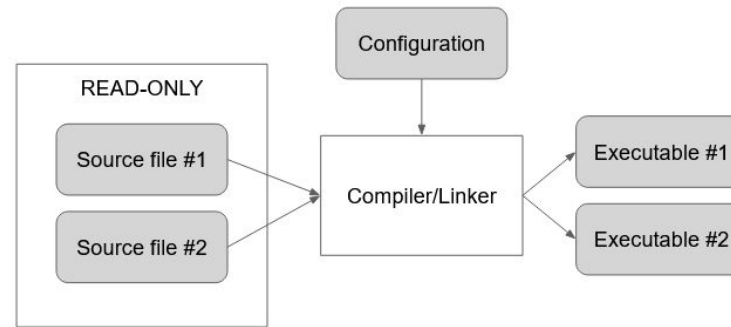
```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```



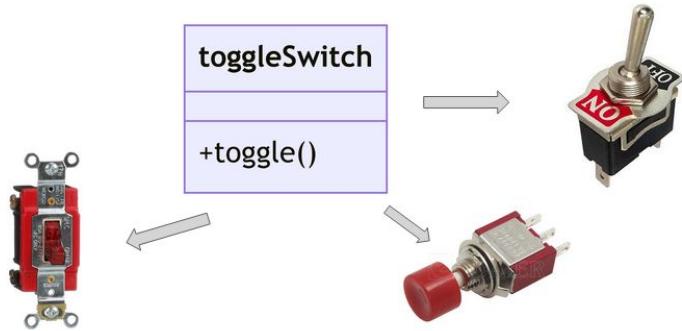
(Build-time)
Polymorphism



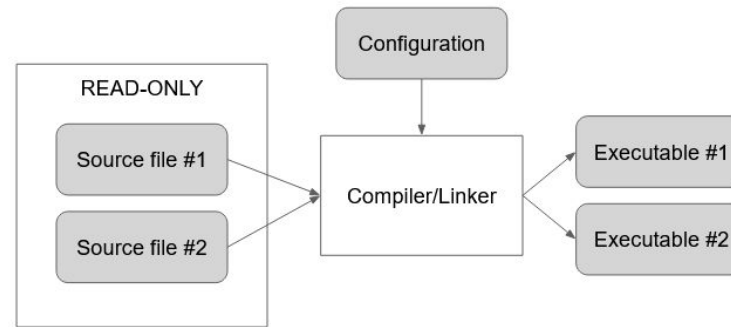
Inheritance

Why and What is it?

Abstraction/ Encapsulation



(Build-time) Polymorphism

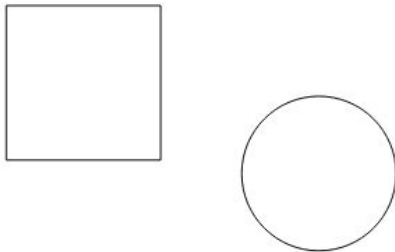


(Run-time) Polymorphism

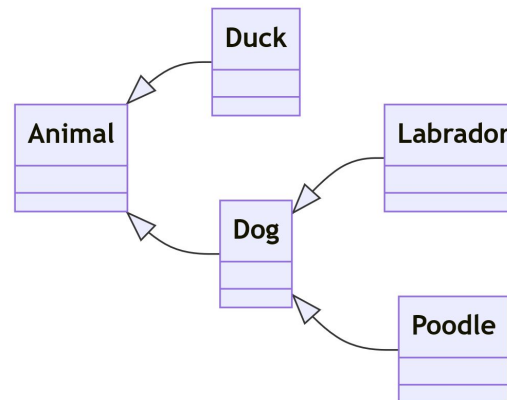
```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```



Inheritance

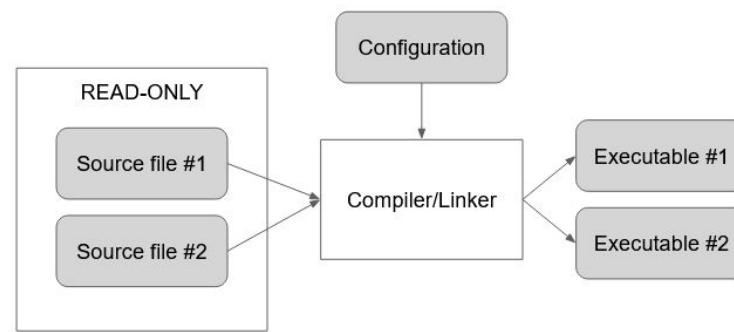
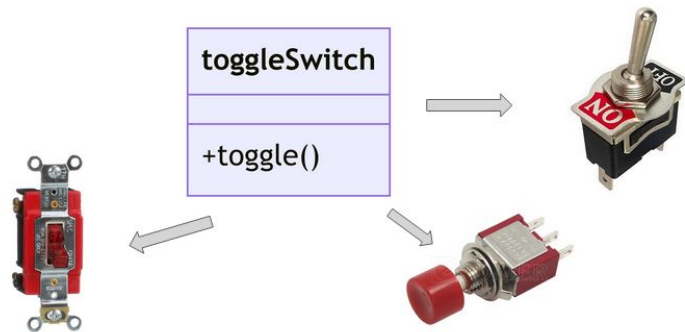


Why and What is it?

Modular code

Abstraction/
Encapsulation

(Build-time)
Polymorphism



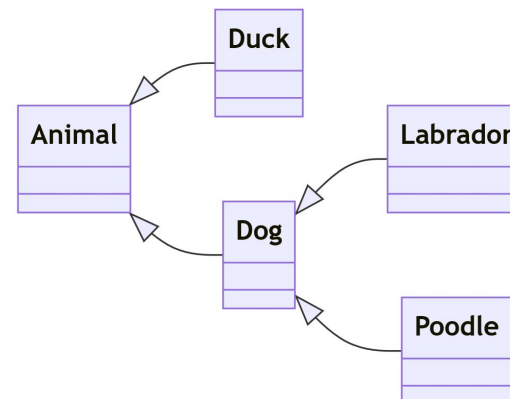
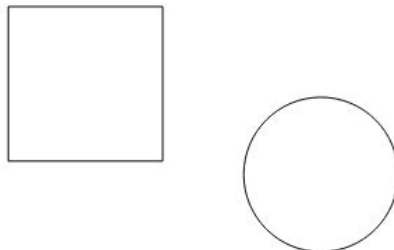
(Run-time)
Polymorphism

Inheritance

```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```



Why and What is it?

Modular code

Modular code is easier to:

- Design/Write
- Change
- Test

Abstraction/
Encapsulation

(Build-time)
Polymorphism

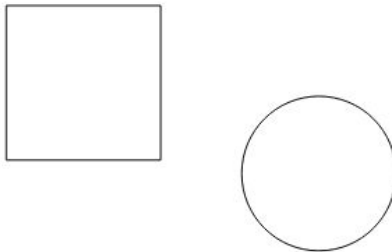


(Run-time)
Polymorphism

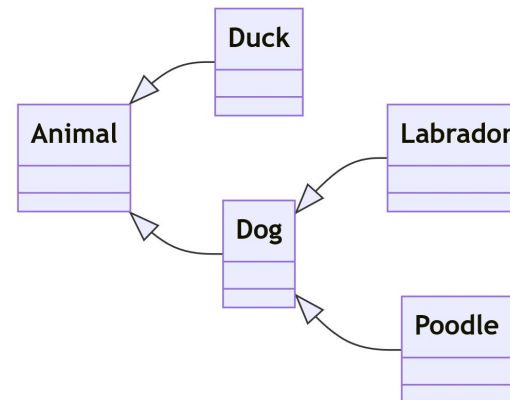
```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```

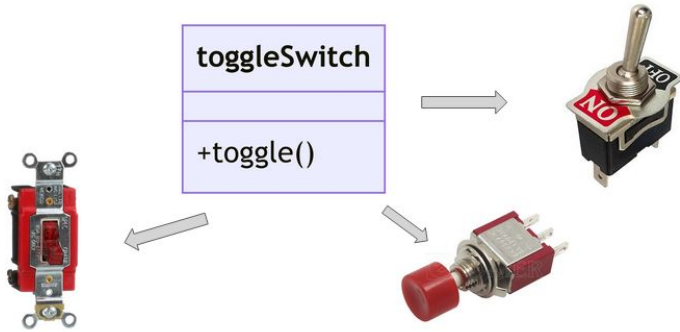


Inheritance

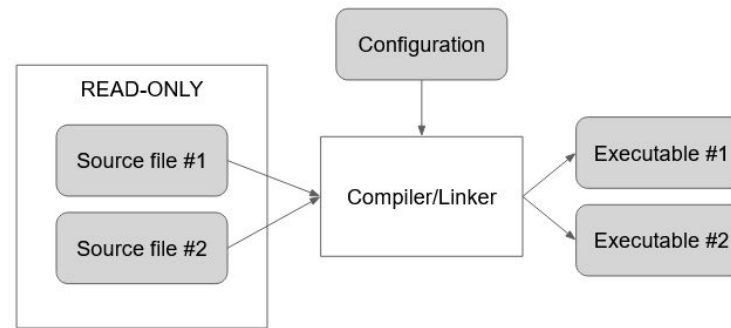


Why and What is it?

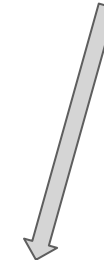
Abstraction/ Encapsulation



(Build-time) Polymorphism



“Type-agnostic” code

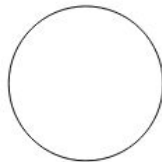


(Run-time) Polymorphism

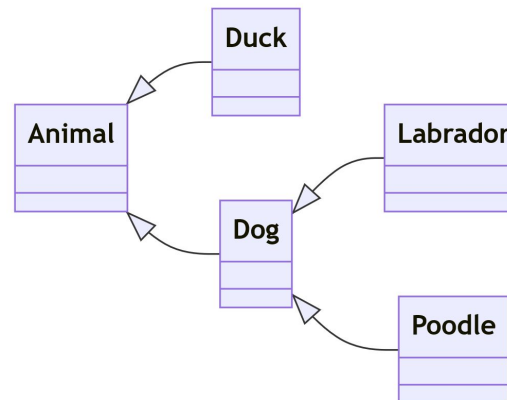
```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```

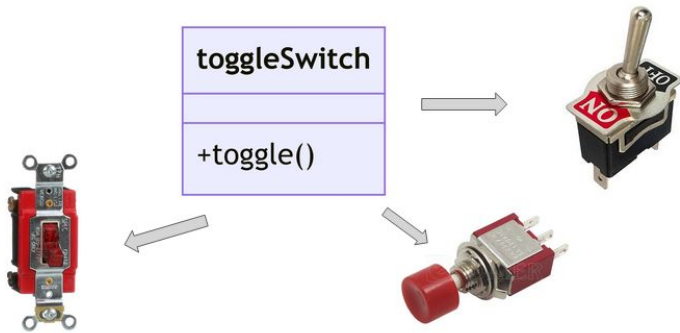


Inheritance

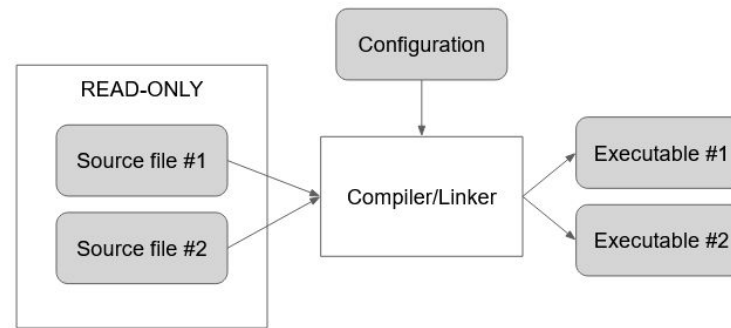


Why and What is it?

Abstraction/ Encapsulation



(Build-time) Polymorphism



“Type-agnostic” code

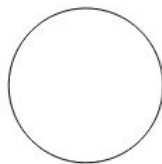
- Encourages code reuse
- Allows for Design Patterns

(Run-time) Polymorphism

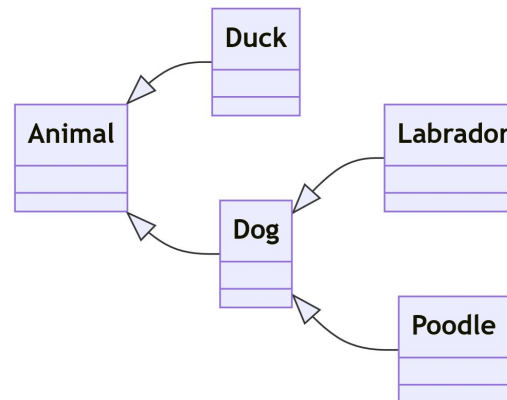
```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```



Inheritance



Why and What is it?

Modular code

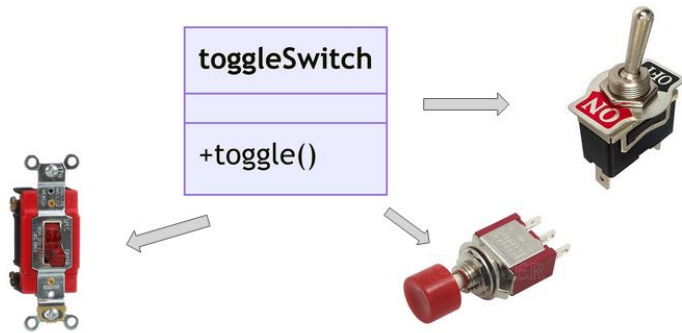
Modular code is easier to:

- Design/Write
- Change
- Test

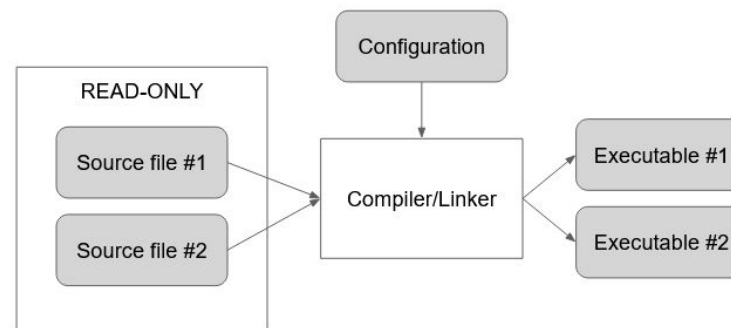
“Type-agnostic” code

- Encourages code reuse
- Allows for Design Patterns

Abstraction/
Encapsulation



(Build-time)
Polymorphism

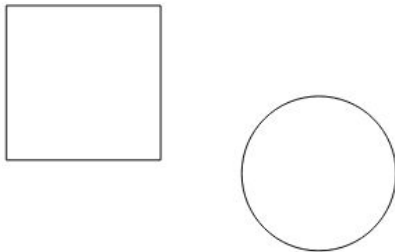


(Run-time)
Polymorphism

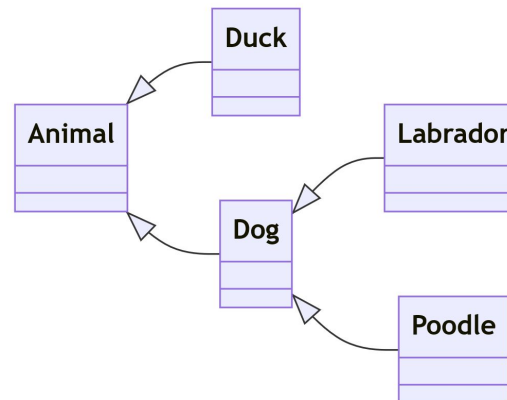
```
p_shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```

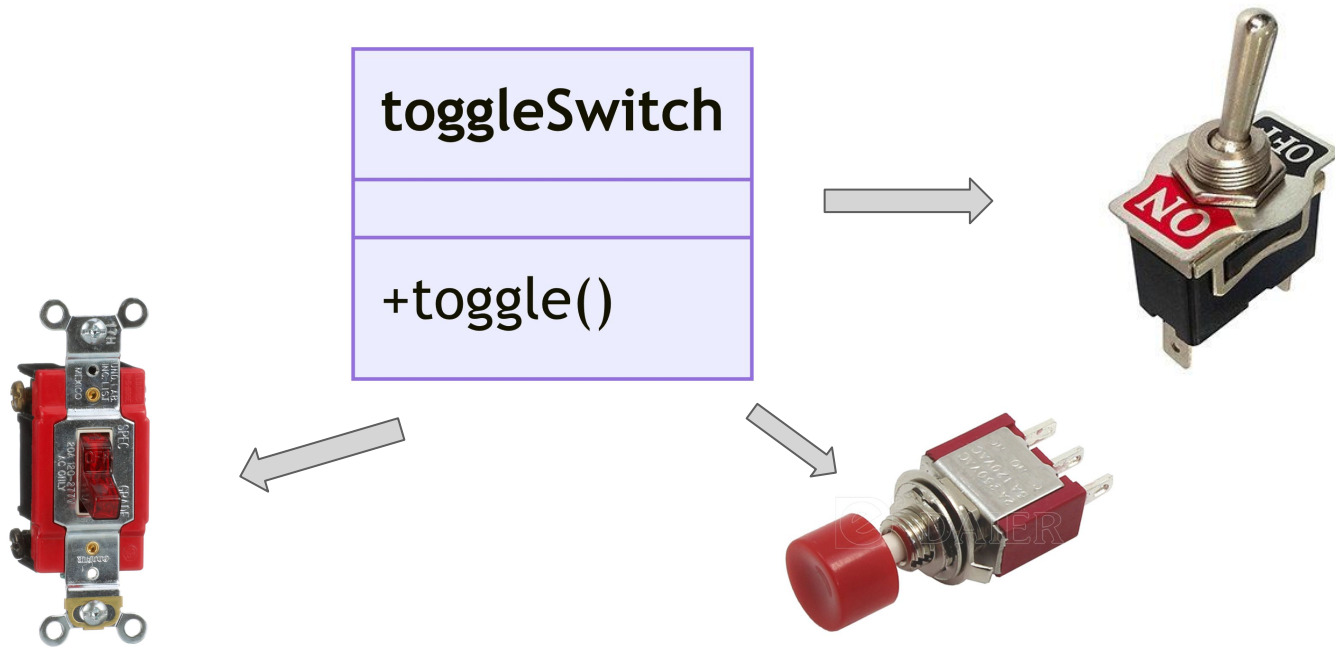


Inheritance



2

Abstraction and Encapsulation



Ducks

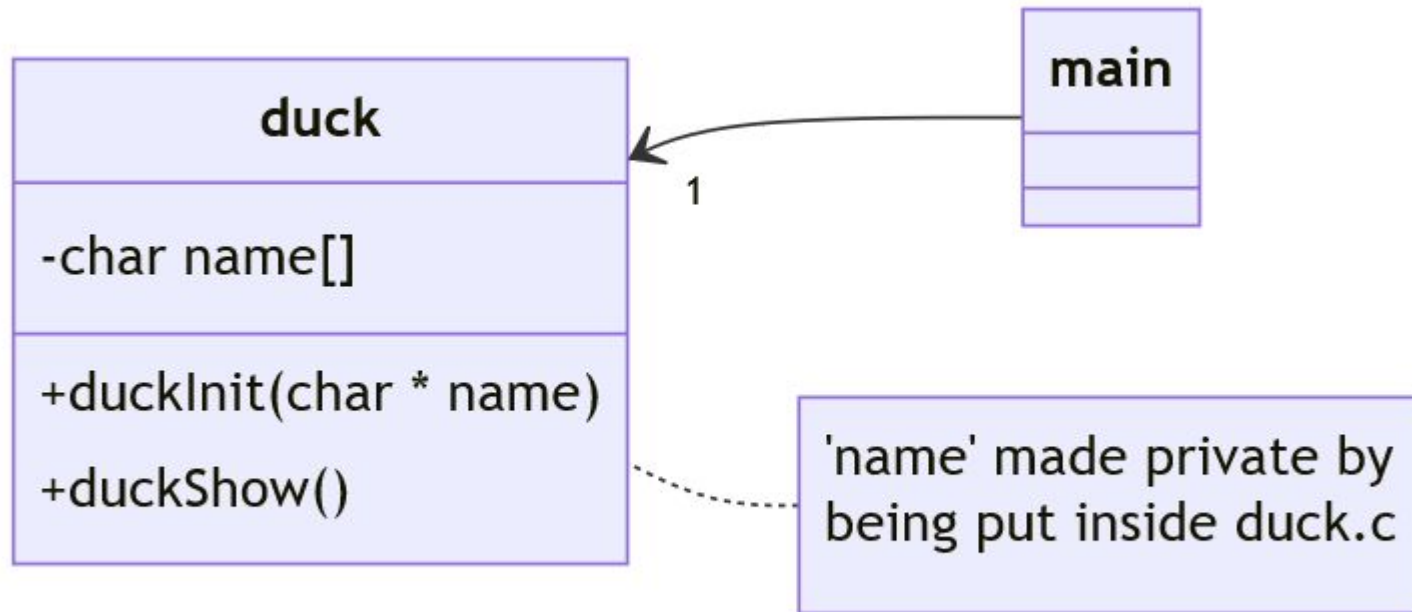
duck

char name[]

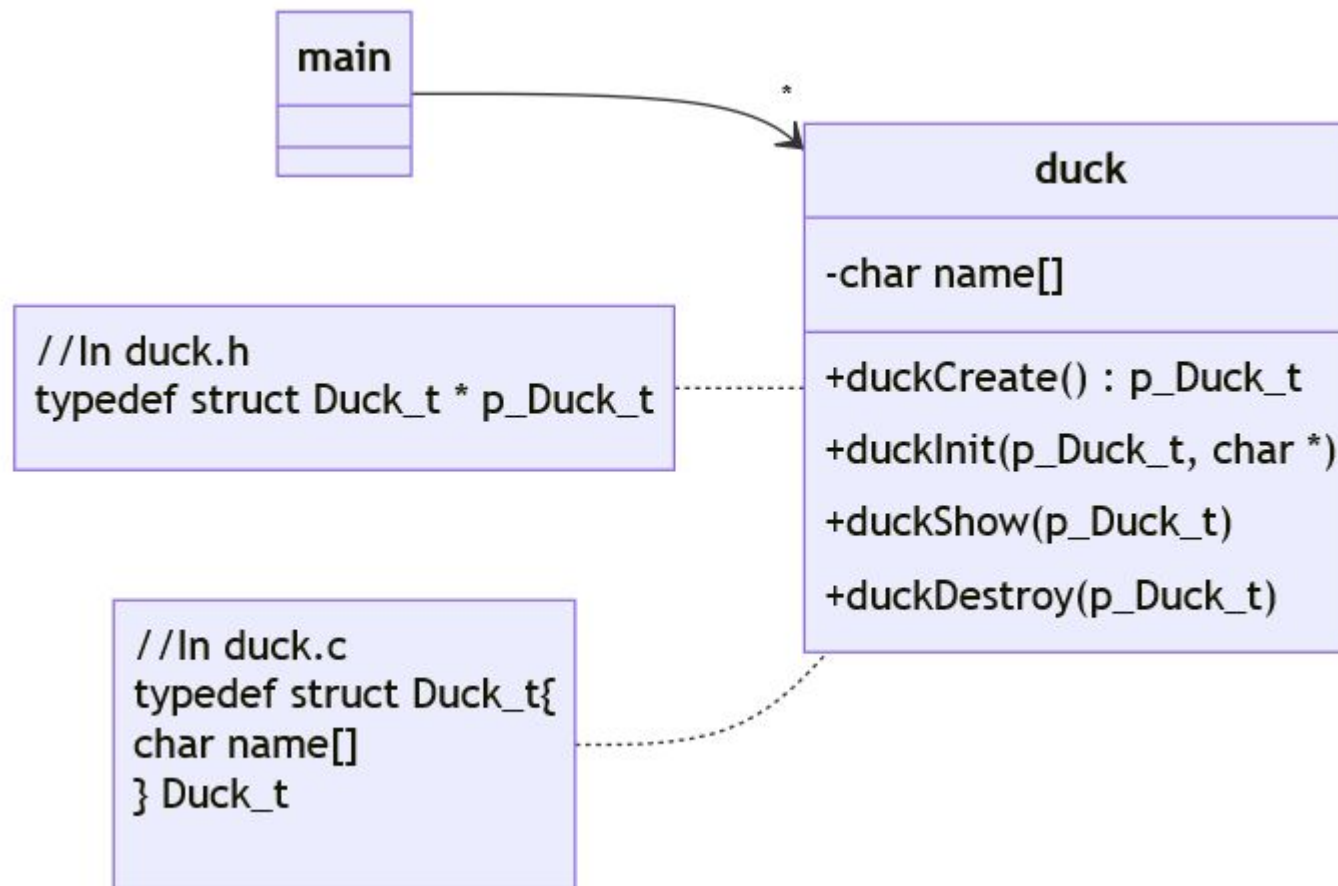
duckInit(char * name)

duckShow()

Singletons



Opaque Structs



Memory Management

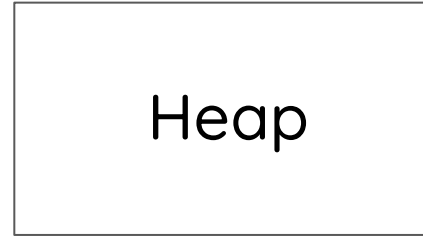
Dynamic

```
p_Duck_t duckCreate( void )  
{
```

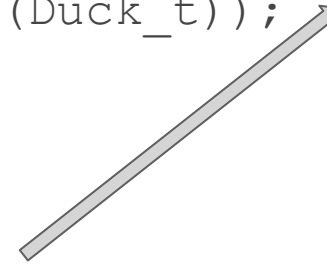
```
    p_Duck_t newDuck = (p_Duck_t) calloc(1, sizeof(Duck_t));  
    return newDuck;  
}
```

Static

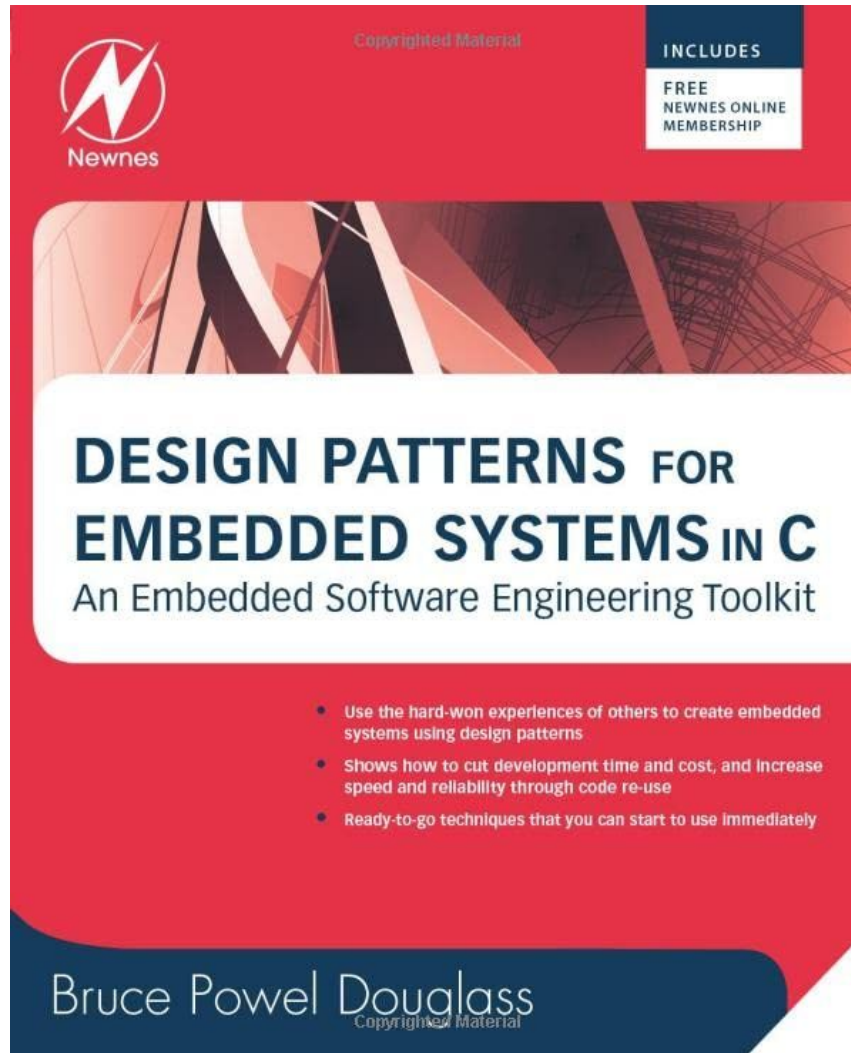
```
// Create array of ducks; i.e. a memory pool  
p_Duck_t duckCreate( void )  
{  
    // newDuck = NULL;  
    // For each element in the array of ducks:  
    // If element is not in use:  
    //     newDuck = element.duck  
    //     break;  
    return newDuck;  
}
```



array[0]	array[1]	...
In use?	In use?	...
struct Duck_t	struct Duck_t	...



Going Further

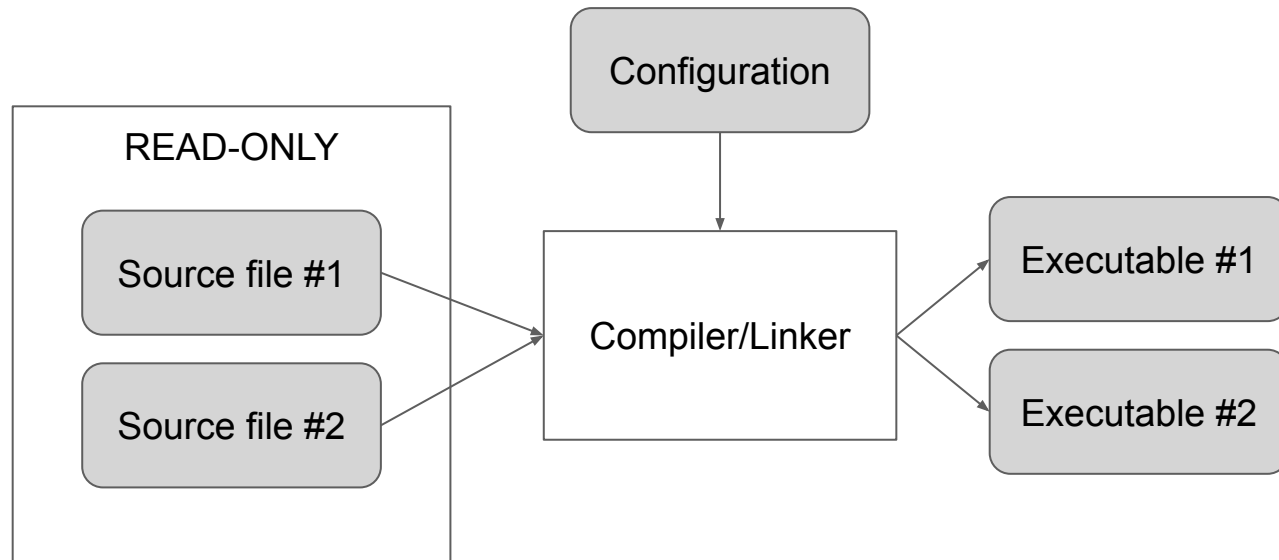


“Comparison of OOP techniques in C”

- Mediator Pattern

3

Build-time Polymorphism



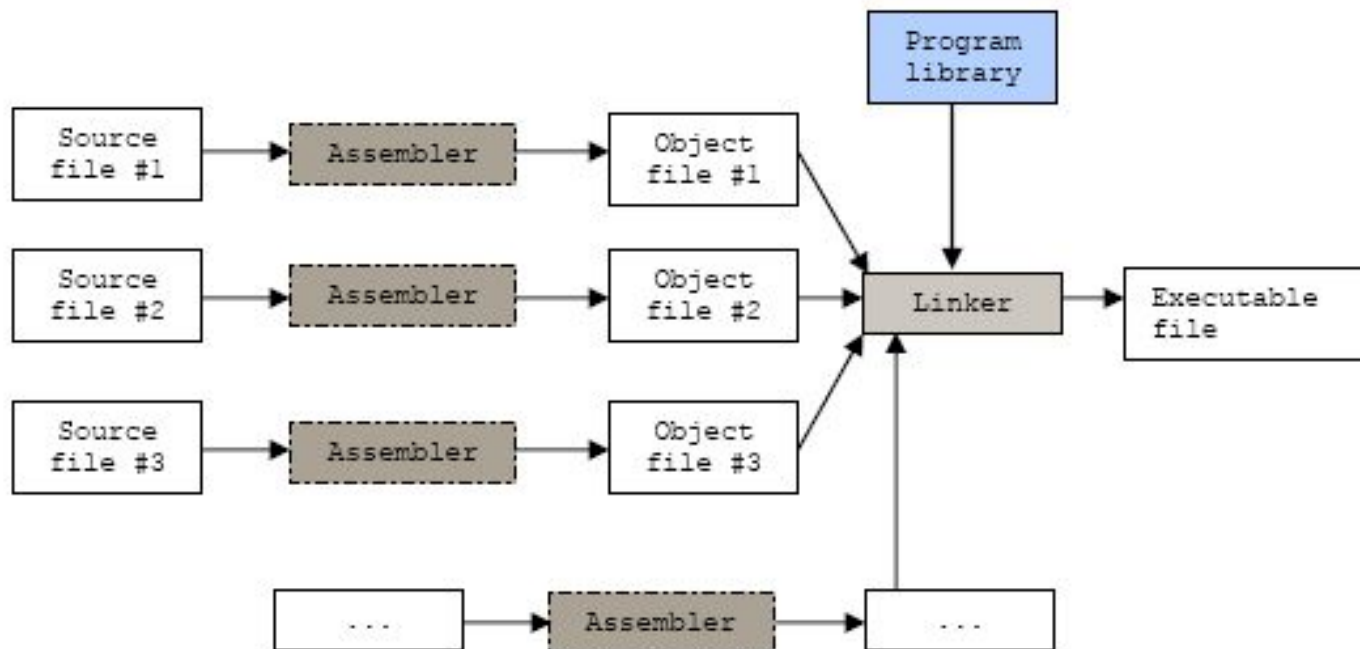
Using the Linker

```
// object.h  
int Obj_Fcn1( int );
```

```
// Source file #1  
#include object.h  
int x = Obj_Fcn1(5);
```

```
// Source file #2  
int Obj_Fcn1( int val )  
{  
    return val + 1;  
}
```

```
// Source file #3  
int Obj_Fcn1( int val )  
{  
    return val + 2;  
}
```



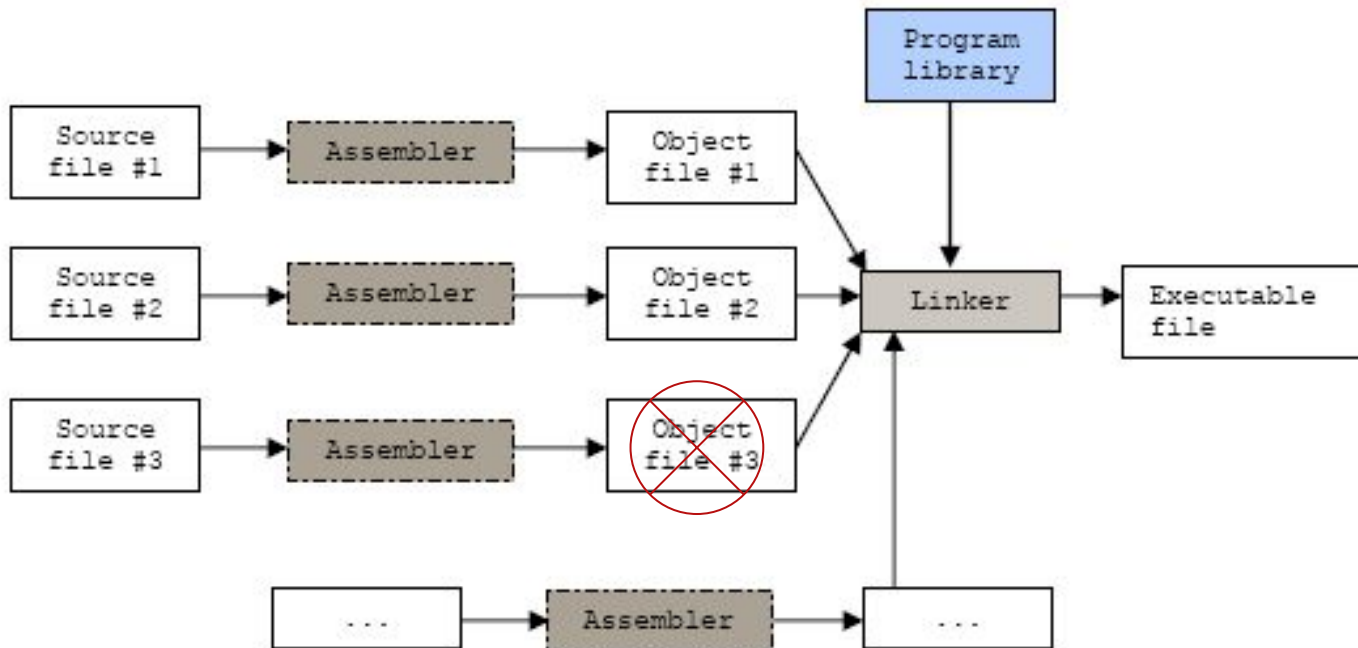
Using the Linker

```
// object.h  
int Obj_Fcn1( int );
```

```
// Source file #1  
#include object.h  
int x = Obj_Fcn1(5); // x gets 6
```

```
// Source file #2  
int Obj_Fcn1( int val )  
{  
    return val + 1;  
}
```

```
// Source file #3  
int Obj_Fcn1( int val )  
{  
    return val + 2;  
}
```



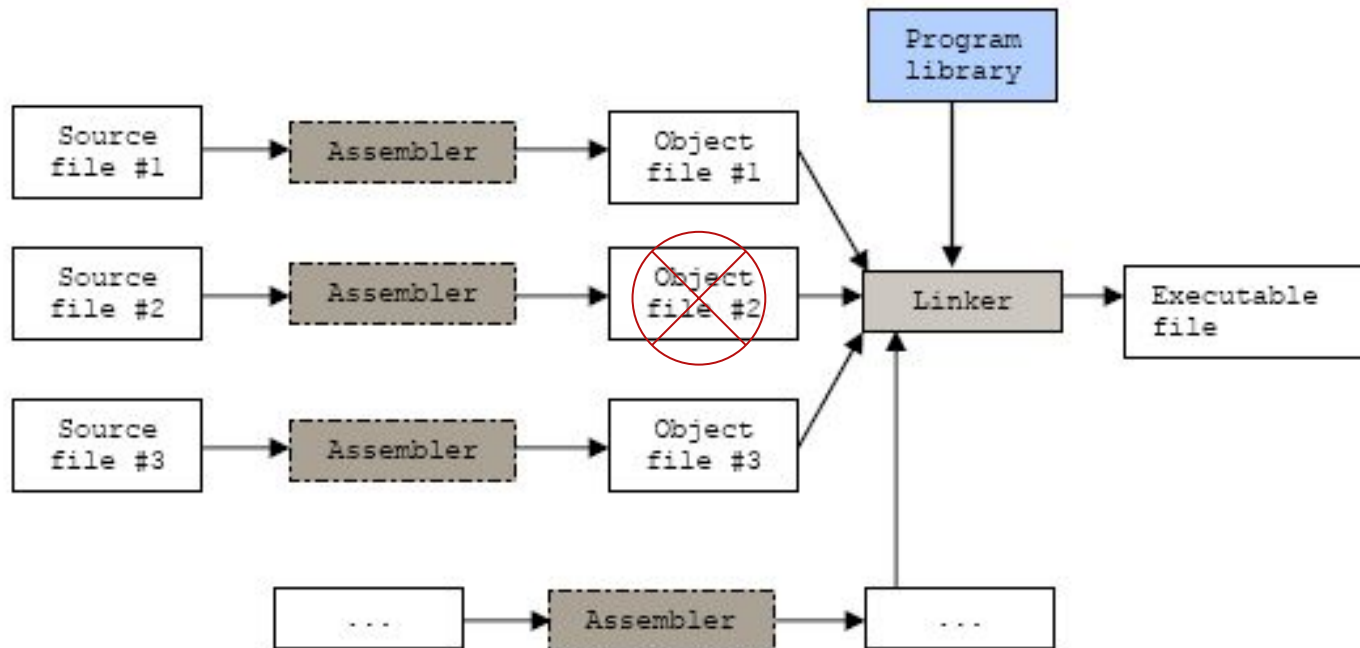
Using the Linker

```
// object.h  
int Obj_Fcn1( int );
```

```
// Source file #1  
#include object.h  
int x = Obj_Fcn1(5); // x gets 7
```

```
// Source file #2  
int Obj_Fcn1( int val )  
{  
    return val + 1;  
}
```

```
// Source file #3  
int Obj_Fcn1( int val )  
{  
    return val + 2;  
}
```

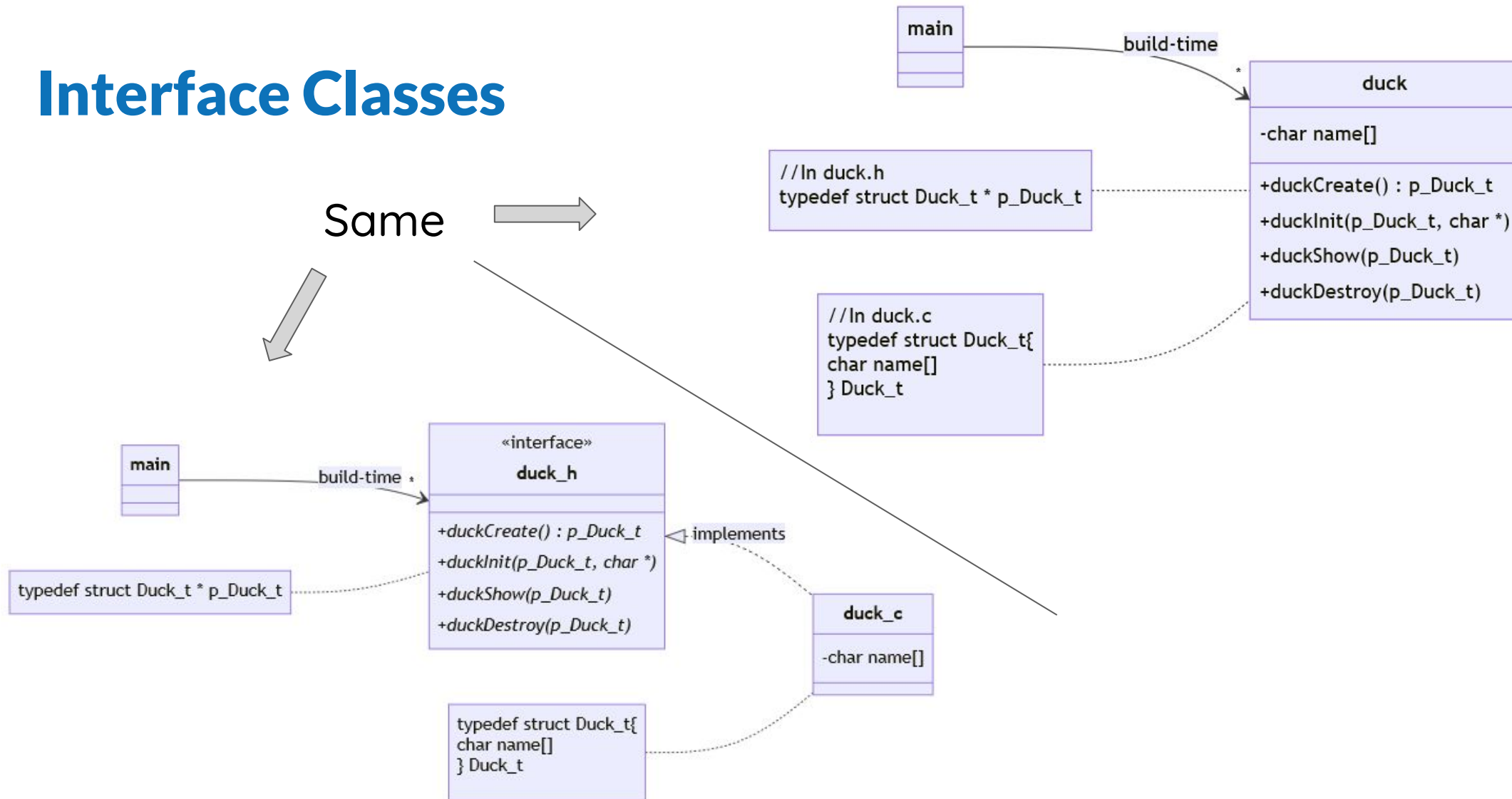




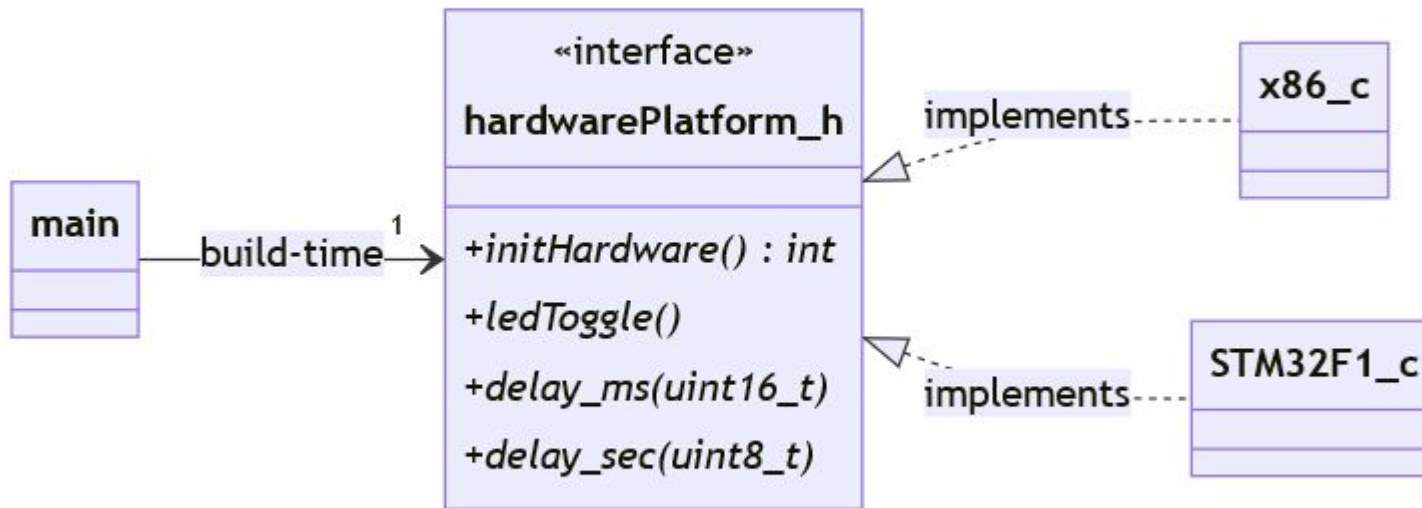
Functional Simulator

Using linker substitution to simulate an embedded system on a laptop or desktop computer.

Interface Classes

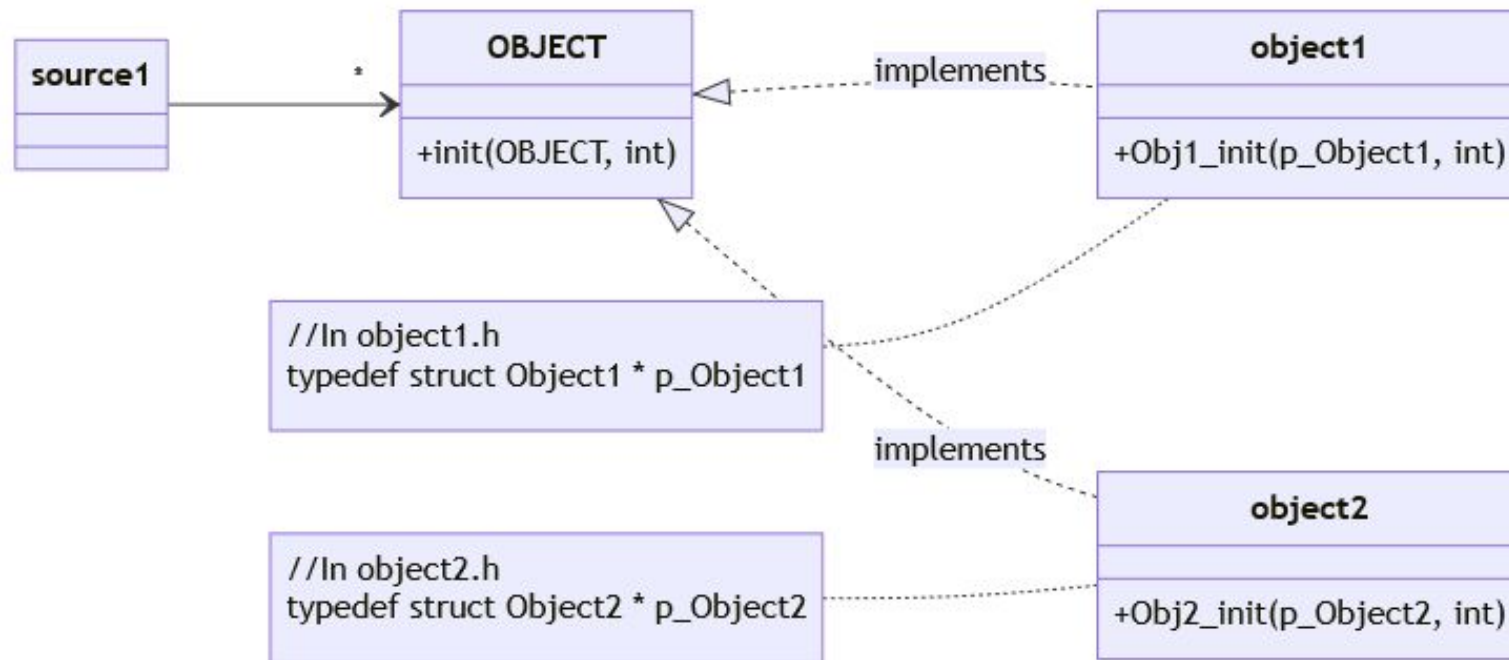


Functional Simulator



<https://github.com/nathancharlesjones/Generic-Makefile-based-Project-for-x86-and-STM32F1>

Using the Preprocessor



Using the Preprocessor

```
// source1.c
```

```
#ifndef OBJ1
#    include object1.h
#    define OBJECT p_Object1
#    define OBJ(fcn, ...) Obj1_## fcn(__VA_ARGS__)
#else
#    include object2.h
#    define OBJECT p_Object2
#    define OBJ(fcn, ...) Obj2_## fcn(__VA_ARGS__)
#endif

OBJECT newObject;           // New generic object
OBJ(init, newObject, 5);    // Generic "init" function
```

Using the Preprocessor

```
// source1.c
```

```
#define OBJ1
```

```
#ifndef OBJ1
```

```
#    include object1.h
```

```
#    define OBJECT p_Object1
```

```
#    define OBJ(fcn, ...) Obj1_## fcn(__VA_ARGS__)
```

```
#else
```

```
#    include object2.h
```

```
#    define OBJECT p_Object2
```

```
#    define OBJ(fcn, ...) Obj2_## fcn(__VA_ARGS__)
```

```
#endif
```

```
OBJECT newObject;          --> p_Object1 newObject;
```

```
OBJ(init, newObject, 5);   --> Obj1_init(newObject,5) ;
```

Using the Preprocessor

```
// source1.c
```

```
#define OBJ2
```

```
#ifndef OBJ1
```

```
#    include object1.h
```

```
#    define OBJECT p_Object1
```

```
#    define OBJ(fcn, ...) Obj1_## fcn(__VA_ARGS__)
```

```
#else
```

```
#    include object2.h
```

```
#    define OBJECT p_Object2
```

```
#    define OBJ(fcn, ...) Obj2_## fcn(__VA_ARGS__)
```

```
#endif
```

```
OBJECT newObject;          --> p_Object2 newObject;
```

```
OBJ(init, newObject, 5);   --> Obj2_init(newObject,5) ;
```

Comparison of Polymorphic Techniques

	Link-time	Compile-time
Difficulty	Easiest	Medium
Scoping	Applies to the entire project	Applies to a single source file

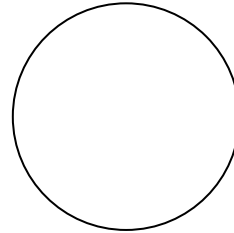
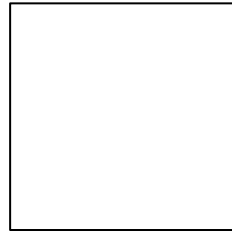
4

Run-time Polymorphism

```
p_Shape_t myShapes[] = { square1, circle1 };
```

```
draw(myShapes[0]);
```

```
draw(myShapes[1]);
```



Function Pointers

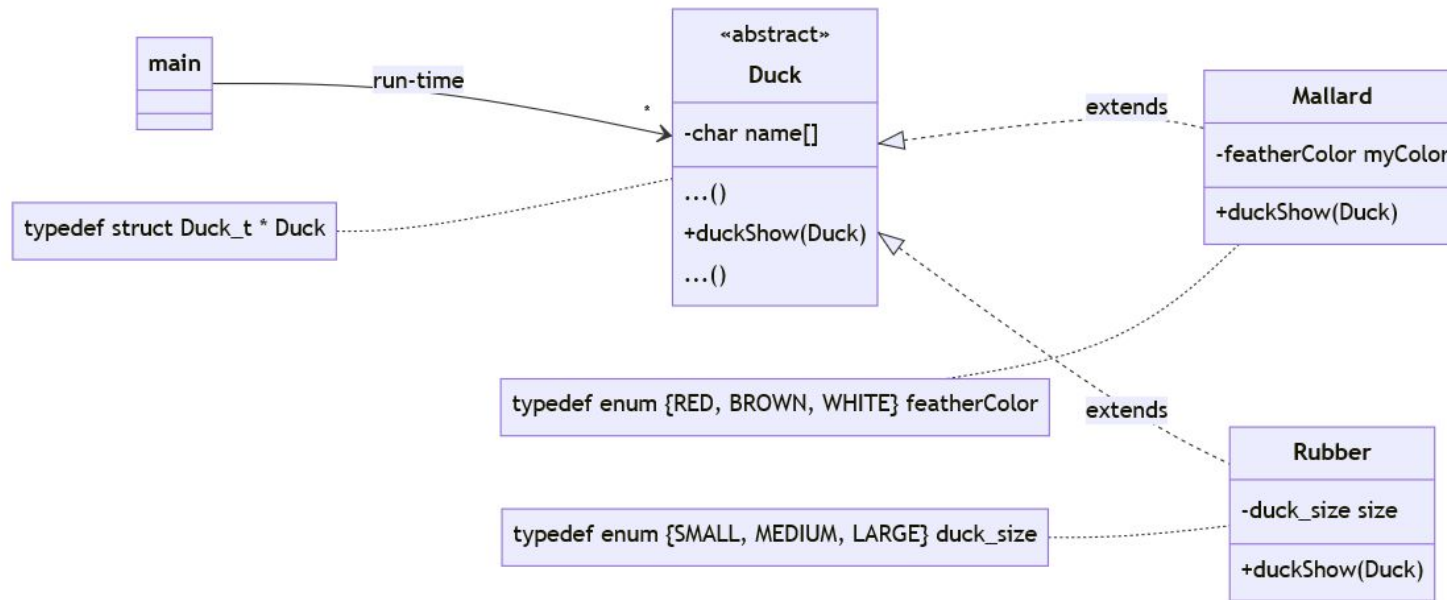
```
void (*change_this_function)(void);

void print_hello()
{
    printf("Hello!\n");
}

void print_goodbye()
{
    printf("Goodbye!\n");
}

void main(void)
{
    change_this_function = print_hello;
    change_this_function();
    change_this_function = print_goodbye;
    change_this_function();
}
```

More Ducks



```
Duck George = /* Make a rubber duck */ ;
Duck Bill = /* Make a mallard duck */ ;
```

```
duckShow(George);
// "Hi! I'm a medium rubber duck. My name is George."
```

```
duckShow(Bill);
// "Hi! I'm a mallard duck. My name is Bill. I have brown feathers."
```

Four Types of Inherited Functions

Final

```
void baseFcn1( p_Obj this )  
{  
    // Base class implementation  
}
```

Virtual

```
void baseFcn1( p_Obj this )  
{  
    if( this->Fcn1 ) this->Fcn1(this);  
    else /* Base class implementation */ ;  
}
```

Four Types of Inherited Functions

Pure Virtual

```
void baseFcn1( p_Obj this )
{
    ASSERT(this->Fcn1);
    this->Fcn1(this);
}
```

Final + Virtual

(a.k.a the Template Method Pattern)

```
void baseFcn1( p_Obj this )
{
    // Base class implementation
    if( this->Fcn1 ) this->Fcn1(this);
}
```

Public, Private, Protected Functions

Public

```
// Obj.h  
void Fcn1(p_Obj);
```

```
// Obj.c
```

```
void Fcn1(p_Obj){...}
```

Private

```
// Obj.h  
// Nothing
```

```
// Obj.c
```

```
static void Fcn1(p_Obj){...}
```

Protected

```
// Obj.h & Obj.c  
// Nothing
```

```
// Obj_private.h
```

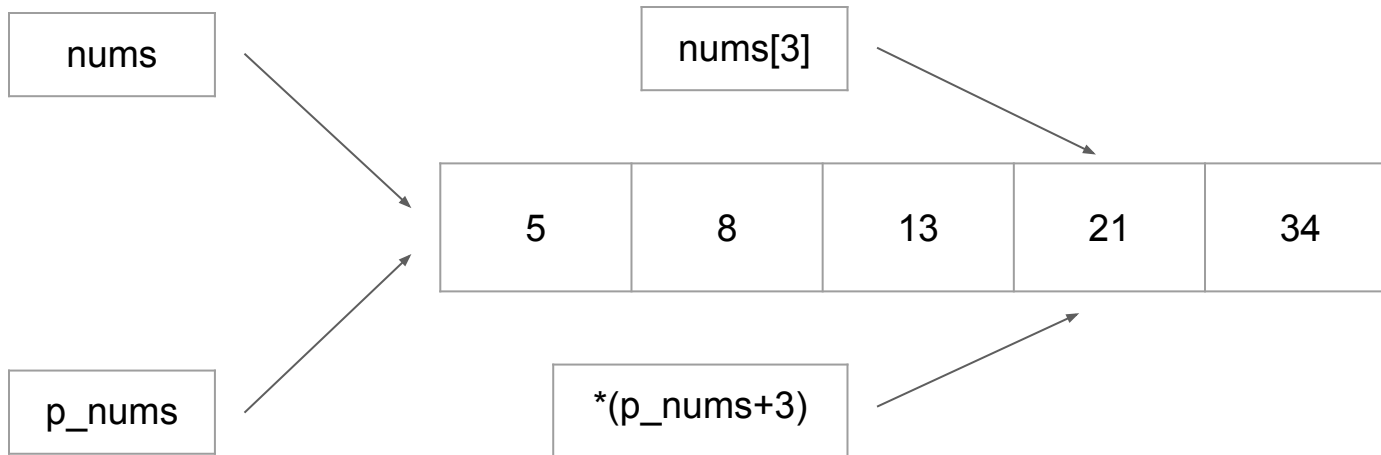
```
#define Fcn1(x) ...
```

```
or
```

```
static inline void Fcn1(p_Obj){...}
```

Different Pointers to the Same Data

```
int nums[5] = {5, 8, 13, 21, 34};  
int *p_nums = nums;
```



Memory Layout of a Duck

```
typedef struct Duck_t {  
    void (*duckShow)( p_Duck_t thisDuck );  
    char name[MAX_CHARS_NAME_WITH_NUL];  
} Duck_t;
```



*duckShow	name									

Different Pointers to the Same Data

```
typedef struct Mallard_t {
    Duck_t parentDuck;
    featherColor myColor;
} Mallard_t, *Mallard;
```

***Mallard is a *Duck**



Mallard											
Duck_t											featherColor
*duckShow	name										
mallardShow	B	i	l	l							BROWN

Interface, Abstract, and Base Classes

Interface Class

```
// No constructor
// No member variables
```

```
// Obj.h
int  f( void );
void g( int );
```

```
// Obj.r
typedef struct Obj
{
    int  (*f) ();
    void (*g) (int);
}
```

Abstract Class

```
// No constructor
// Member variables
```

```
// Obj.h
int  f( void );
void g( int );
```

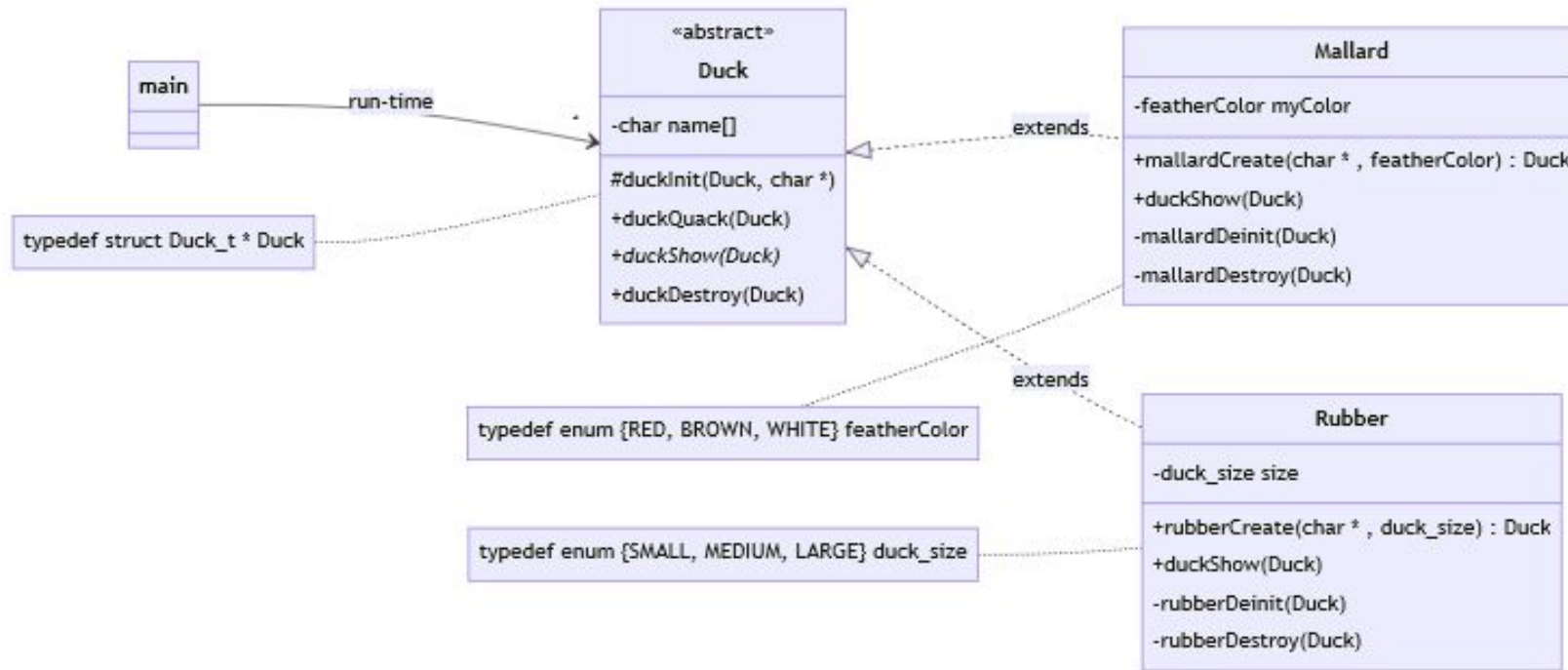
```
// Obj.r
typedef struct Obj
{
    int  (*f) ();
    void (*g) (int);
    int  val1;
    float dbl1;
}
```

Base Class

```
// Constructor
```

```
// Obj.h
p_Obj new( void );
```

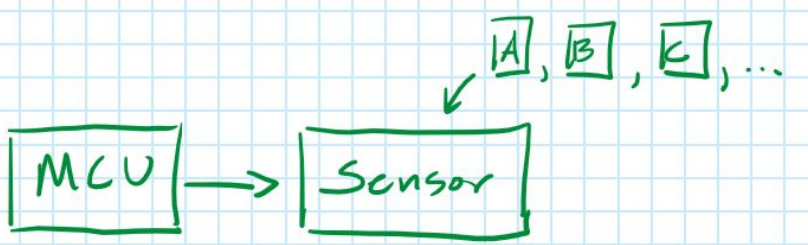
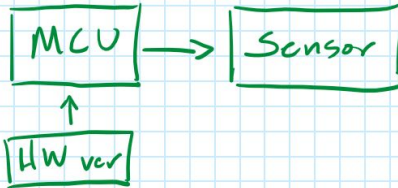
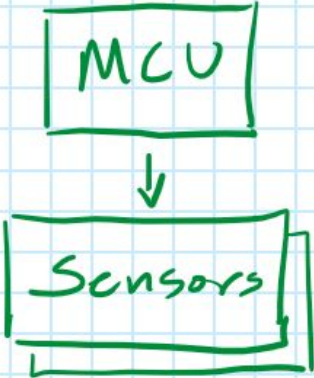
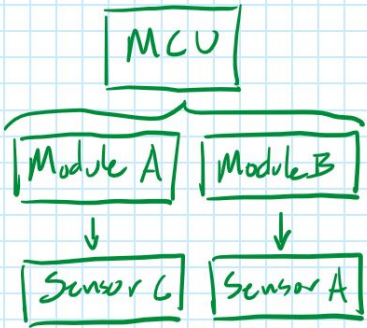
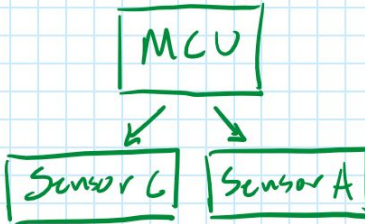
More Ducks



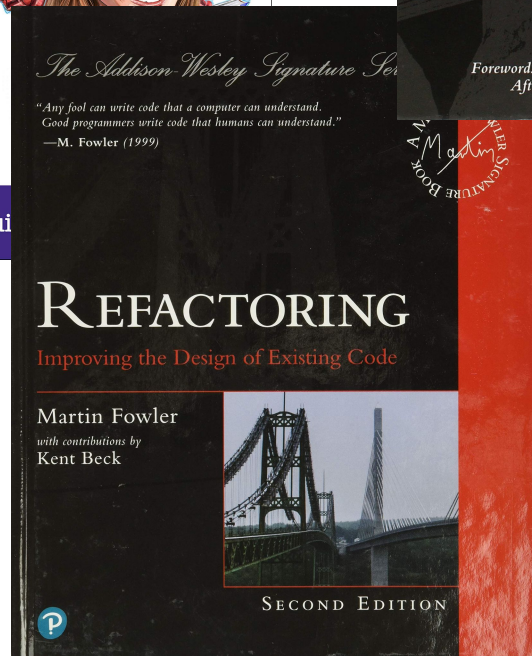
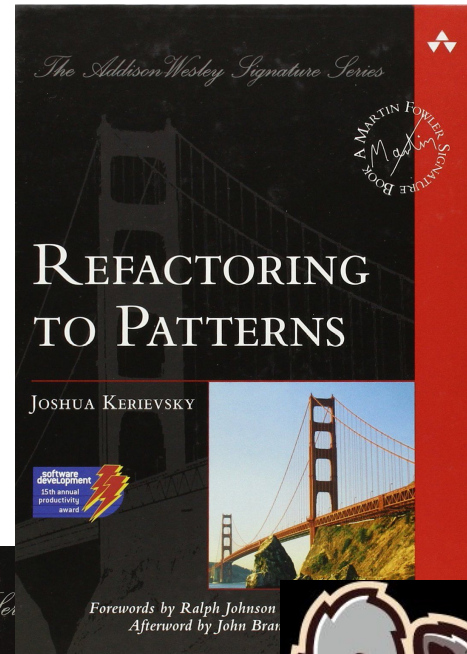
Comparison of Polymorphic Techniques

	Link-time	Compile-time	Run-time
Difficulty	Easiest	Medium	Hardest
Scoping	Applies to the entire project	Applies to a single source file	Applies to each individual object

Keep it Simple, Student

	Link-time	Compile-time	Run-time
Singleton			
Opaque struct			

Going Further

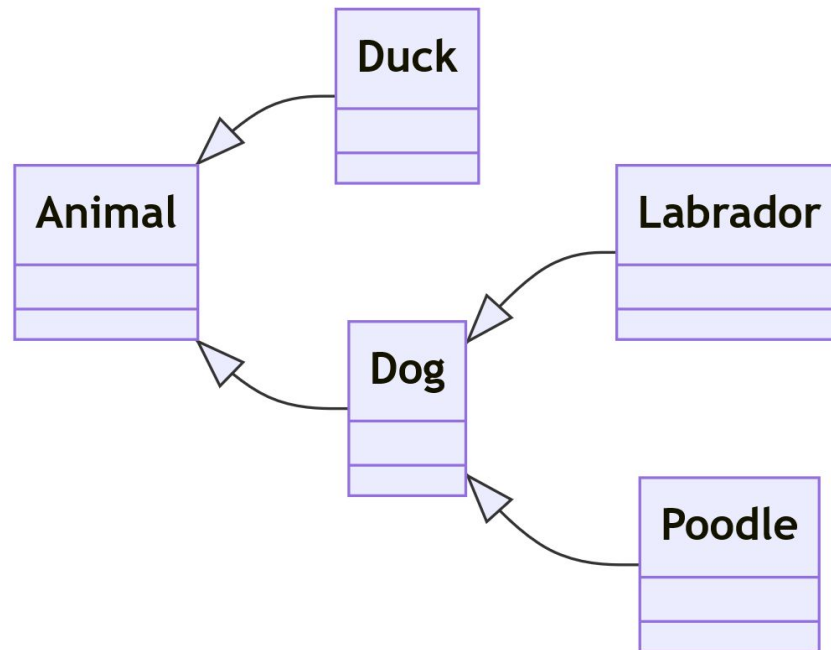


“Comparison of OOP techniques in C”

- Template Method Pattern
- Factory Pattern

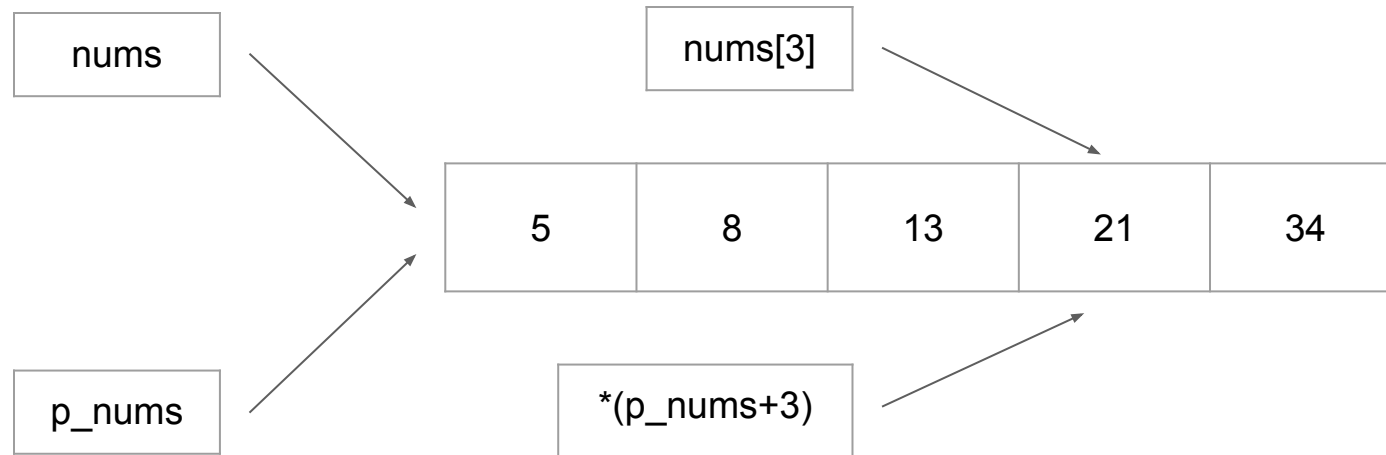
5

Inheritance



Matroska Structs

```
int nums[5] = {5, 8, 13, 21, 34};  
int *p_nums = nums;
```



Matroshka Structs

*redMallard_t is a
*Mallard_t is a *Duck_t

redMallard_t												
Mallard_t												Other data
Duck_t										featherColor		
vtable	name											
*	B	i	l	l							BROWN	...



*show	*deinit	*destroy
-------	---------	----------



Matroshka Structs

redMallard interface is a
Mallard interface is a ...



redMallard_t													
Mallard_t												Other data	
Duck_t										featherColor			
vtable	name												
*	B	i	l	l							BROWN	...	

↓

*show	*deinit	*destroy	*migrate	*Other fcn
Duck interface				
Mallard interface				
redMallard interface				



void *

```
Duck duckNew(...);
```

```
Duck mallardNew(...);
```

```
??? redMallardNew(...):
```

void *

~~Duck~~ duckNew (...);

~~Duck~~ mallardNew (...);

~~???~~ redMallardNew (...);

Duck duckNew (...);

Mallard mallardNew (...);

redMallard redMallardNew (...);

duckShow ((Duck) aMallard);

void *

```
Duck duckNew(...);  
Duck mallardNew(...);  
??? redMallardNew(...);
```

```
void * duckNew(...);  
void * mallardNew(...);  
void * redMallardNew(...);
```

```
duckShow(aMallard);
```

```
// duck.c  
void duckShow( void * _thisDuck )  
{  
    Duck thisDuck = (Duck)_thisDuck;  
    ...  
}
```

Two Problems

1. Is “_thisDuck” *really* even a Duck??

```
Car myCar = (Car)carNew(...);
```

```
duckShow(myCar);
```

Two Problems

1. Is “_thisDuck” *really* even a Duck??

```
Car myCar = (Car)carNew(...);
```

```
duckShow(myCar);
```

=> Objects point to base class

Two Problems

1. Is “_thisDuck” *really* even a Duck??

```
Car myCar = (Car)carNew(...);  
duckShow(myCar);
```

=> Objects point to base class

2. Is “_thisDuck” *really* even an object??

```
int x = 5;  
duckShow(&x);
```

Two Problems

1. Is “_thisDuck” *really* even a Duck??

```
Car myCar = (Car)carNew(...);  
duckShow(myCar);
```

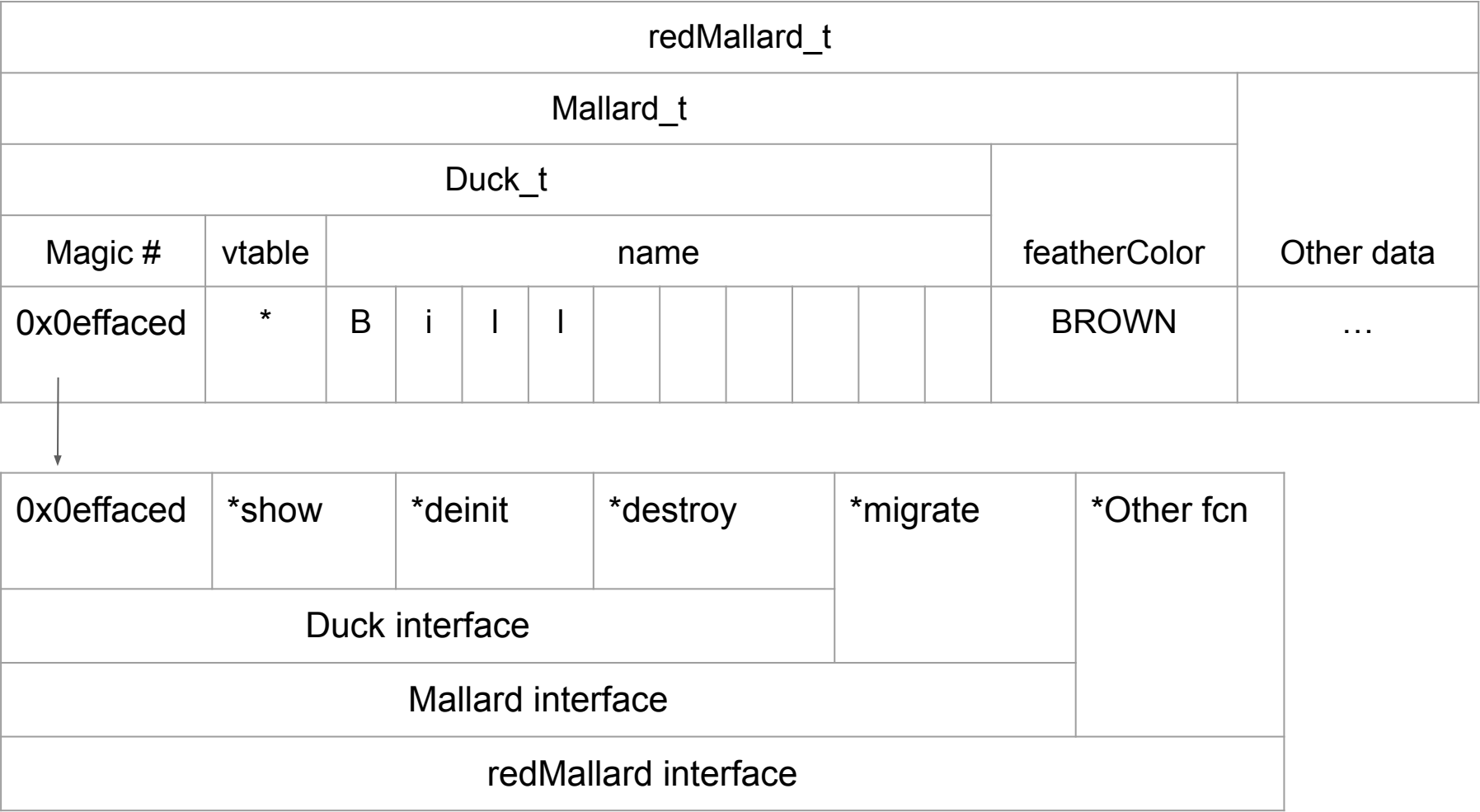
=> Objects point to base class

2. Is “_thisDuck” *really* even an object??

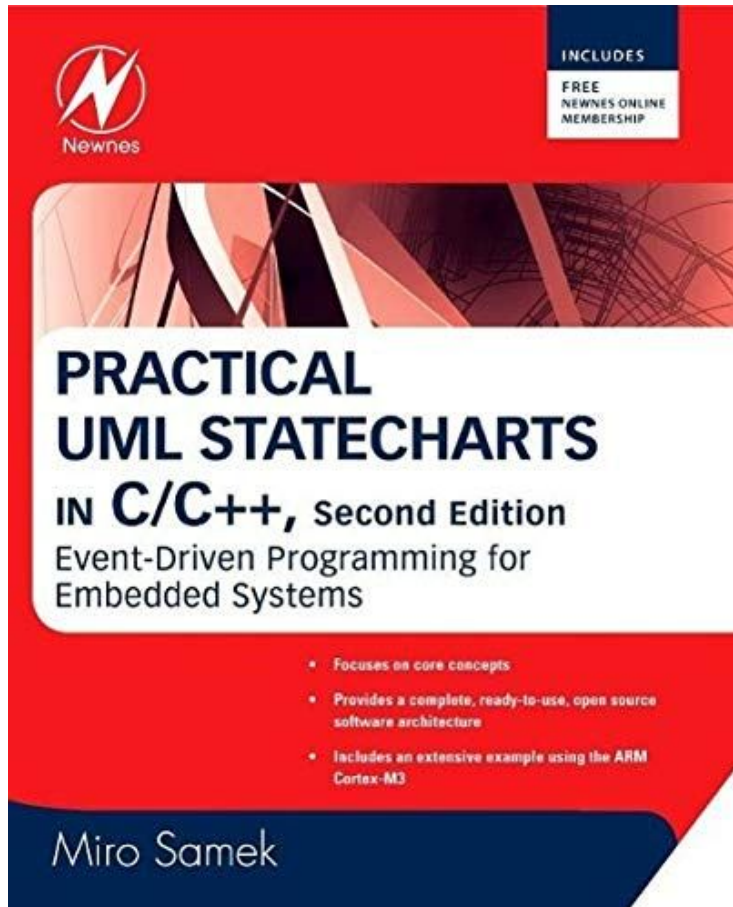
```
int x = 5;  
duckShow(&x);
```

=> Magic numbers

Magic Numbers



Going Further



*Free PDF

“OOC” by Axel Schreiner

1

1 Abstract Data Types Information Hiding

1.1 Data Types

Data types are an integral part of every programming language. ANSI-C has **int**, **double** and **char** to name just a few. Programmers are rarely content with what's available and a programming language normally provides facilities to build new data types from those that are predefined. A simple approach is to form aggregates such as arrays, structures, or unions. Pointers, according to C. A. R. Hoare "a step from which we may never recover," permit us to represent and manipulate data of essentially unlimited complexity.

What exactly is a data type? We can take several points of view. A data type is a set of values — **char** typically has 256 distinct values, **int** has many more; both are evenly spaced and behave more or less like the natural numbers or integers of mathematics. **double** once again has many more values, but they certainly do not behave like mathematics' real numbers.

Alternatively, we can define a data type as a set of values plus operations to work with them. Typically, the values are what a computer can represent, and the operations more or less reflect the available hardware instructions. **int** in ANSI-C does not do too well in this respect: the set of values may vary between machines, and operations like arithmetic right shift may behave differently.

More complicated examples do not fare much better. Typically we would define an element of a linear list as a structure

```
typedef struct node {
    struct node * next;
    ... information ...
} node;
```

and for the operations we specify function headers like

```
node * head (node * elt, const node * tail);
```

This approach, however, is quite sloppy. Good programming principles dictate that we conceal the representation of a data item and declare only the possible manipulations.

1.2 Abstract Data Types

We call a data type *abstract*, if we do not reveal its representation to the user. At a theoretical level this requires us to specify the properties of the data type by mathematical axioms involving the possible operations. For example, we can remove an element from a queue only as often as we have added one previously, and we retrieve the elements in the same order in which they were added.

Summary

01

Abstraction/ Encapsulation

- Singleton
- Opaque structs

02

Polymorphism

- Linker substitution
- Preprocessor
- Function pointers

03

Inheritance

- Matroshka structs
- Manual type system

Examples

- Mediator
- Functional simulator
- Template method
- Factory

THANK YOU

Embedded
Online
Conference

www.embeddedonlineconference.com